



Optimized surface discretization of functionally defined multi-material objects

Pierre-Alain Fayolle^{a,*}, Alexander Pasko^b

^aThe University of Aizu, Aizu-Wakamatsu city, Fukushima 965-8580, Japan

^bBournemouth University, Poole, UK

ARTICLE INFO

Article history:

Received 21 February 2011

Received in revised form 7 September 2011

Accepted 11 October 2011

Available online 17 November 2011

Keywords:

Implicit surface

Function representation

Meshing

Mesh optimization

Delaunay triangulation

Multi-material object

Heterogeneous object

ABSTRACT

We present in this paper an algorithm for meshing implicit surfaces based on the Delaunay triangulation of a point-set adaptively sampled on an implicit surface. To improve the quality of the resulting triangular mesh, we use at each iteration a mesh optimization algorithm with the following objectives: optimizing the connectivity, retrieving the sharp features, regularizing the triangles shapes and minimizing the approximation error. Then, we extend this algorithm in order to handle functionally defined heterogeneous object surfaces, while maintaining a good quality for the triangles' shapes and the mesh features (geometrical sharp features and boundaries between different materials).

© 2011 Elsevier Ltd. All rights reserved.

1. Introduction

Implicit surfaces [1,2] represent surfaces by continuous sets of points in space taking a given function value (isovalue), for example the surface defined by the isovalue 0 of f in R^3 is: $\partial S = \{\mathbf{p} \in R^3: f(\mathbf{p}) = 0\}$. Replacing $=$ by $\geq$ in the previous equation defines more generally a solid instead of a surface. This representation is popular because of its ability to describe complicated shapes and to handle easily topological changes during simulation [3].

In this work, models for complex objects are built step by step in a constructive manner from simple primitives with predefined implicit surfaces by applying set operations (union, intersection, difference) to the primitives. Set operations can be implemented by using min/max or R -functions (see for example [1,2,4]). The resulting complex objects are described by the zero level-set of a single function.

Visualization of such objects can be done by ray-tracing or by polygonization [5]. Polygonization consists in generating an approximation of the surface $f = 0$ by a discrete set of triangles. It is usually done by sampling the function on a regular grid and inspecting each cell in the grid to check if it intersects with the surface. Intersection between a cell and the surface is done by looking at the sign of the function values at each corner of the cell.

The problem we address in this work is to obtain an approximation of an implicit surface, which may be a boundary of a multi-material heterogeneous object, by a good quality optimized triangular mesh. The generated mesh should satisfy the following criteria: regularized shape and size of mesh triangles, optimized connectivity, retrieved sharp features, minimized approximation error (to the implicit surface) and mesh edges aligned with material boundaries for multi-material objects. Such meshes are required in object analysis using boundary-element and finite-element methods, object fabrication and other critical applications.

1.1. Related works

1.1.1. Marching cubes and variants

The marching cubes algorithm was first introduced by Lorensen and Cline in [5]. It is a popular algorithm for extracting a triangulated surface from a regular grid but suffers from several drawbacks: topological ambiguities, exhaustive enumeration of cells, badly shaped triangles or destroyed sharp features among others. A survey of the MC algorithm and its extensions was recently presented by Newman and Yi [6].

Natarajan [7] and Chernyaev [8] identify the problem of internal ambiguity and provide solutions. Lewiner et al. [9] provide an efficient implementation of Chernyaev [8]. Another approach for solving topological ambiguities is given in [10].

An algorithm for retrieving sharp features of the surface was proposed by Kobbelt et al. [11]. Their algorithm extends the MC algorithm by using the local distance field information and its gradient to compute and insert into the mesh additional sample points

* Corresponding author.

E-mail addresses: fayolle@u-aizu.ac.jp (P.-A. Fayolle), apasko@bournemouth.ac.uk (A. Pasko).

lying on the surface's features. Ohtake et al. [12,13] use a set of post-processing steps to improve the triangular mesh generated by the MC algorithm. Their work attempts to handle sharp features correctly as well as to regularize the mesh by vertex relocation.

In general, these algorithms tend to produce an excessive number of triangles, especially in the regions of low curvature, because of their use of a uniform grid. Moreover, the shapes of the triangles are not controlled and can degenerate even when a post-processing optimization is used.

Methods have been developed to apply the MC algorithm to an adaptive grid in [14,15]. However cracks appearing when two cells of different resolutions meet need to be patched as done in [16].

The Dual Contouring algorithm introduced by Ju et al. [17] can also be used for extracting surfaces with adaptive resolution while reproducing sharp features (using Hermite data). The original Dual Contouring algorithm does not guarantee intersection-free surfaces and may have holes due to topological errors. It was later extended by Ju and Udeschi [18] to solve this issue. Ho et al. propose in [19] to convert the 3D marching cubes into 2D cubical marching squares. This algorithm generates surfaces adaptively without crack patching, maintains a consistent topology and sharp features. In these methods, there is no mechanism to control the shape of the triangles.

Improving the quality of marching cubes meshes is done in [20] by transforming the edges, intersecting the implicit surface, to places that improve the mesh quality. In [21,22], a new classification is presented for the MC case tables based on the different configurations of the edges intersecting the implicit surface within a cell. A modified MC case table is proposed based on this edge group formulation that yields significantly improved meshes compared to MC algorithms. Raman and Wenger propose a related approach in [23], however their method may generate non-manifold meshes and tends to change the mesh topology.

1.1.2. Isosurface approximation methods

Surface tracking methods start from a seed triangle on the surface and grow by iteratively adding new triangles to approximate the surface. Tracking algorithms can be traced to the work of Wyvill et al. [24]. In the Marching Triangles algorithm [25], the decision of adding a new triangle is based on the Delaunay surface constraint proposed by Boissonnat [26]. The original Marching Triangles algorithm suffers from cracks and is extended by Akkouche and Galin [27] to overcome this issue. In [28], Schreiner et al. propose an advancing front technique that uses a guidance field to generate sample on the surface. The choice of seed points is important for these tracking methods as there can be only as many connected components as seed points. In addition, none of these algorithms seem to be able to handle properly sharp features of the surface.

In the algorithm proposed in [29] by Desbrun et al., seed points on the bounding volume of an implicit surface migrate to the surface to generate a mesh approximating the surface. The algorithm is fast and allow interactive update of the mesh when adding or removing primitives. However, it is designed to work with implicit surfaces defined by skeletal elements only and it may fail to tessellate correctly regions where many primitives blend.

In [30], De Figueiredo et al. propose to sample the implicit surface with a particle system. Particles tend to be attracted by area of high curvatures. The sampling is then made uniform by introducing repelling forces between each particles and solving until equilibrium. An improved algorithm for sampling particles on an implicit surface is described by Witkin and Heckbert in [31]. A triangular mesh approximating the implicit surface is obtained from a Delaunay triangulation of the particles. Meyer et al. propose in [32] an algorithm that similarly samples particles on the implicit surface using a sizing field and then distributes them. The resulting point-set is then meshed using a Delaunay based algorithm. If the initial set of particles is not properly scattered, these algorithms may miss

disjoint pieces and holes of the surface. These sampling approaches rely on heavy processing and are time consuming. Finally, sharp features are not handled properly by these algorithms.

1.1.3. Delaunay refinement

Chew describes in [33] an algorithm that constructs a restricted Delaunay triangulation of a surface by iteratively inserting the furthest intersection of surface with all Voronoi edges. Boissonnat and Oudot give in [34] an algorithm, which iteratively adds the centers of the bad surface Delaunay balls to the current point-set, until all Delaunay balls become good. Whether a surface Delaunay ball is good or not is decided based on the ratio of the circumradius with a 1-Lipschitz function. In their analysis, the distance to the medial axis is used for the 1-Lipschitz function, which is rather difficult to compute practically. Dey et al. propose a related algorithm in [35] but the topological guarantee between the output mesh and the input surface is obtained by using a theorem by Edelsbrunner and Shah [36] on the topological ball property. They also employ techniques to limit the number of computations of the Delaunay triangulation. These methods do not capture sharp edges. Similarly to tracking methods, they also require some sample points on each connected component of the surface.

Gelas et al. propose in [37] an algorithm related to [35] with an additional post-processing step used for capturing sharp edges and regularizing the mesh following the re-meshing techniques of Valette and Chassery [38].

1.1.4. Polygonization of heterogeneous objects

The marching cubes algorithm has been extended by Nielson and Franke to handle multi-material instead of binary values in [39]. In their approach, each cell of a regular grid is first decomposed in tetrahedrons. They identify four cases of tetrahedrons (depending on the material index at each node) and show how to divide them to compute the material interface. Bloomenthal and Ferguson proposed another approach as part of their algorithm for polygonizing non-manifold implicit surfaces [40]. Suzuki et al. [41] proposed another extension of the marching cubes algorithm to handle multi-material objects by analyzing the possible patterns of n ($n \in 1 \dots 8$) distinct materials in one cell. All these approaches are based on the marching cubes algorithm and therefore inherit its weaknesses. In their paper introducing the dual contouring algorithm [17], Ju et al. also describe an extension for multi-material modeling, which is similar to the approaches mentioned above [39–41]. This extension has naturally the same problems as the Dual Contouring algorithm. Shammaa et al. introduce in [42] a method for the segmentation of multi-material CT data in order to extract boundary surface. In their approach, multi-material contouring is done by a technique similar to the one presented in [17].

Meyer et al. [43] recently extended the meshing algorithm proposed in [32] for meshing segmented 3D images. Their algorithm is designed for medical images and does not seem to handle sharp features. The initial sampling of the surface relies on heavy processing.

1.2. Overview and main contributions

In this paper, we present new algorithms for the generation of high-quality surface meshes from implicit surfaces. First a polygonization algorithm using adaptively sampled points on an implicit surface is proposed. Starting from a small sample of random points, mesh vertices are iteratively added where the surface approximation is insufficient, while the associated triangle mesh is optimized.

This algorithm is simple to implement, gives a direct control on the number of vertices on the mesh, handles sharp features and generates well shaped triangles. In contrast to marching cubes related

algorithms, the function is sampled only at the vertices and not at each cell corners (including empty cells away from the surface).

We extend this new algorithm for discretizing the surface of heterogeneous objects (objects made of multiple materials). Given a heterogeneous object defined by a set of functions (to define the object and material partitions geometries), we propose an algorithm that: (1) discretizes each material partition surface and (2) assembles them together while protecting the material boundaries. Resulting triangular meshes exhibit good quality while maintaining sharp features of the objects and boundaries between multiple materials.

2. Adaptive polygonization of implicit surfaces

In this section, we describe our polygonization algorithm based on the Delaunay triangulation of a set of points adaptively sampled on an implicit surface (see Algorithm 1). Details corresponding to each step are given in the following subsections.

Algorithm 1 computes a polygonal approximation of the surface corresponding to the zero level-set of a function $f: \mathbb{R}^3 \rightarrow \mathbb{R}$. At first (lines 1–2), a small number of random points is created and each point is projected on the surface $f=0$. In all our experiments, 5 random points are used to initialize the algorithm. We extend this point-set by additional points located at the nodes of regular grids

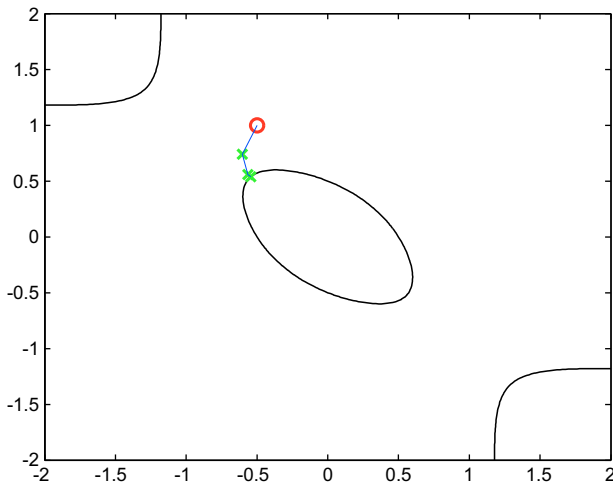


Fig. 1. Error committed when projecting a point on an implicit surface with a first order projection. The original point is represented by the circle. The projected points after one, two and three projections are represented by a cross. The implicit curve is defined by: $x^2 + y^2 + xy - 0.5x^2y^2 - 0.25 = 0$.

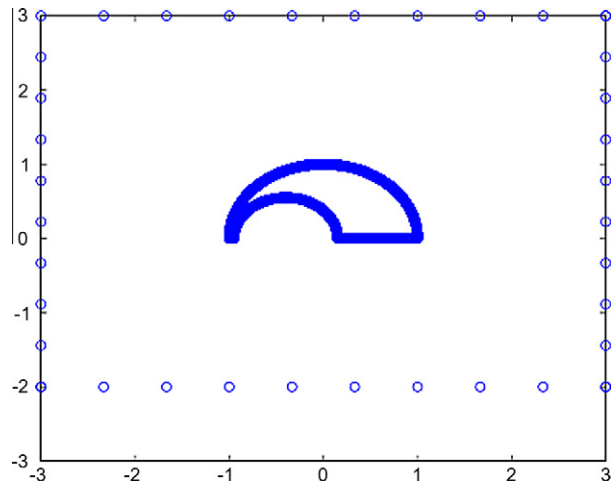


Fig. 2. Points are added on the faces of a regular grid enclosing the object.

on each face of a bounding box enclosing the initial random points (lines 3–4).

The main part of the algorithm is the loop in lines 5–16 where vertices are adaptively added to the triangle mesh and the mesh is optimized. A polygonal approximation of the surface is obtained by computing the Delaunay triangulation of the point-set P , keeping the faces of the triangulation corresponding to the surface boundary $f=0$ (lines 6–11). The mesh is optimized by a series of steps: extracting sharp features, regularizing the mesh and maintaining the vertices close to the surface (line 12). At each loop iteration, some of the tetrahedrons' circumcenters (line 8) and triangles' centroids (line 13) are selected, projected on the surface $f=0$ and added to a list of possible additional vertices. If the number of vertices is still below the user specified number, we add extra vertices in area of high curvature (line 17–22).

Finally, the mesh is further optimized by a series of edge flipping and edge collapsing in order to improve the mesh quality (line 23).

Algorithm 1. Adaptive algorithm for the polygonization of an implicit surface

Require: A function $f: \mathbb{R}^3 \rightarrow \mathbb{R}$ and a maximum number of points n .

- 1: Generate a small number of random points $P = \{\mathbf{p}_i\}$.
- 2: Project these points $\mathbf{p}_i$ on the surface $f=0$.
- 3: Generate a small number of ghost points $G = \{\mathbf{g}_i\}$.
- 4: Add the ghost points to the original point-set: $P = P \cup G$.
- 5: **while** the number of mesh vertices is less than n **OR** there is no more points to add **do**
- 6: Compute the Delaunay triangulation D_t of the point-set P .
- 7: Remove the tetrahedrons from D_t outside of the solid.
- 8: Select tetrahedron circumcenters $\mathbf{c}_i$ such that $d(\mathbf{c}_i)/r_i < \beta$.
- 9: Project $\mathbf{c}_i$ on the surface $f=0$.
- 10: Add these points to the list of additional points.
- 11: Find in D_t the set of triangles T on the surface.
- 12: Optimize the mesh (T, P) .
- 13: Select triangle centroids $\mathbf{c}_i$ such that $d(\mathbf{c}_i) > h_0$ and $r_i > R_{min}$.
- 14: Project $\mathbf{c}_i$ on the surface $f=0$.
- 15: Add these points to the list of additional points.
- 16: **end while**
- 17: **while** the number of vertices is less than n **do**
- 18: Compute the curvature at each triangle centroid.
- 19: Add the triangles in a priority queue (sorted by curvature).
- 20: Subdivide the first triangle in the queue using a one to four subdivision. Neighboring triangles are subdivided accordingly to avoid cracks and hanging vertices.
- 21: Add the newly generated triangles in the queue.
- 22: **end while**
- 23: Process triangles with a minimum angle below some user defined threshold by edge swapping/collapsing and vertex relocation (see Section 2.8).

2.1. Projection on the implicit surface

Given a point $\mathbf{p}$ and a function f , we are searching for the projection $\mathbf{p}_s$ of the point $\mathbf{p}$ on the implicit surface $f=0$. If f is the signed distance function, then the projection: $\mathbf{p}_s \leftarrow \mathbf{p} - f(\mathbf{p})\nabla f(\mathbf{p})$ is exact (whenever the gradient of f is defined). If f is not the signed

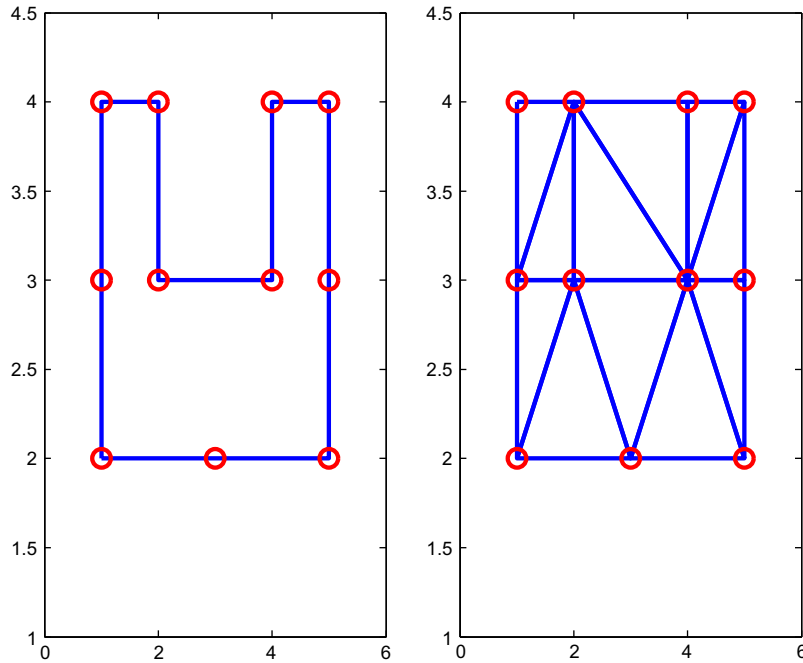


Fig. 3. The Delaunay triangulation of a point-set covers the convex hull. Outside tetrahedrons (triangles in two dimensions) need to be removed to retrieve the boundary of the solid.

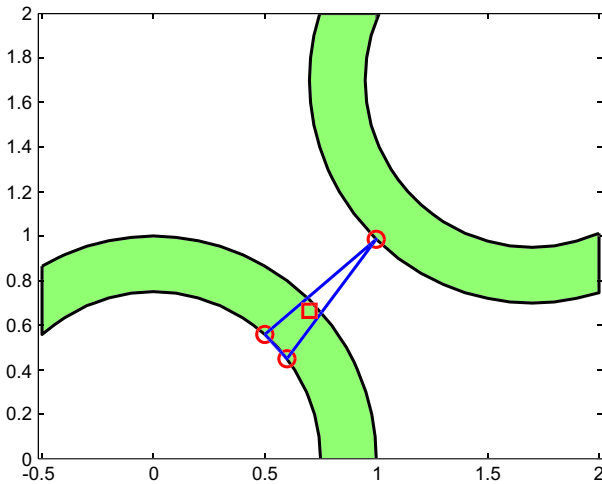


Fig. 4. Due to an insufficient sampling during the first iteration of the algorithm, the surface may cross the inside of a tetrahedron.

distance function, it is possible to project the point on the surface using a numerical algorithm or to approximate the projection.

2.1.1. Projection for general implicit surface

$\Delta \mathbf{p} = \mathbf{p}_s - \mathbf{p}$ is parallel to $\nabla f(\mathbf{p}_s)$ and at $\mathbf{p}_s, f(\mathbf{p}_s) = 0$, leading to the following system of equations:

$$\begin{aligned} f(\mathbf{p} + \Delta \mathbf{p}) &= 0 \\ \Delta \mathbf{p} + t \nabla f(\mathbf{p} + \Delta \mathbf{p}) &= \mathbf{0} \end{aligned} \quad (1)$$

By eliminating t , we can get a system in $\mathbf{p}_s$ only:

$$R(\mathbf{p}_s) = \begin{pmatrix} f(\mathbf{p}_s) \\ f_y(x_s - x) - f_x(y_s - y) \\ f_z(x_s - x) - f_x(z_s - z) \end{pmatrix} = \mathbf{0} \quad (2)$$

where f_x, f_y and f_z are the partial derivatives of f with respect to x, y and z at $\mathbf{p}_s$.

Eq. (2) is solved for the unknown vector column $\mathbf{p}_s$ with the damped Newton method using as initial guess the current point $\mathbf{p}$: let J be the Jacobian matrix of the system $R, \mathbf{p}_{k+1} = \mathbf{p}_k - \alpha J^{-1}(\mathbf{p}_k)R(\mathbf{p}_k)$ is iterated until $R(\mathbf{p}_k) < \epsilon$. The damping parameter α is set to 1.0 in our experiments and ϵ to 10^{-5} . The system in Eq. (2) requires the function f to be smooth ($f \in C^2$).

2.1.2. First order approximation

The previous projection can be time consuming. It is possible to obtain a good approximation of this projection. Starting with the first order Taylor expansion: $f(\mathbf{p}_s) = f(\mathbf{p} + \Delta \mathbf{p}) \approx f(\mathbf{p}) + \Delta \mathbf{p} \cdot \nabla f(\mathbf{p})$, and taking the gradient gives: $\nabla f(\mathbf{p} + \Delta \mathbf{p}) \approx \nabla f(\mathbf{p})$. By combining with $\Delta \mathbf{p} + t \nabla f(\mathbf{p} + \Delta \mathbf{p}) = \mathbf{0}$ (see Eq. (1)), we obtain: $\Delta \mathbf{p} + t \nabla f(\mathbf{p}) = \mathbf{0}$. Combined with $f(\mathbf{p}_s) = 0 \approx f(\mathbf{p}) + \Delta \mathbf{p} \cdot \nabla f(\mathbf{p})$ we obtain:

$$f(\mathbf{p}) - t \|\nabla f(\mathbf{p})\|^2 = 0 \quad (3)$$

and:

$$\Delta \mathbf{p} = -f(\mathbf{p}) \frac{\nabla f(\mathbf{p})}{\|\nabla f(\mathbf{p})\|^2} \quad (4)$$

The first order projection of $\mathbf{p}$ to the implicit surface $f=0$ is thus given by:

$$\mathbf{p} \leftarrow \mathbf{p} - f(\mathbf{p}) \frac{\nabla f(\mathbf{p})}{\|\nabla f(\mathbf{p})\|^2} \quad (5)$$

The above method works if $\nabla f(\mathbf{p})$ exists and if $\nabla f(\mathbf{p}) \neq 0$. For constructive models built using R-functions [1,4] this is the case except for some isolated set of points which are in general avoided by the algorithm since they are on the surface of the solid.

Practically, one iteration of Eq. (5) is not sufficient to reach the surface as illustrated by Fig. 1. We can iterate the projection given in Eq. (5) until $f(\mathbf{p}) < \epsilon$ (in our experiments we used $\epsilon = 10^{-5}$).

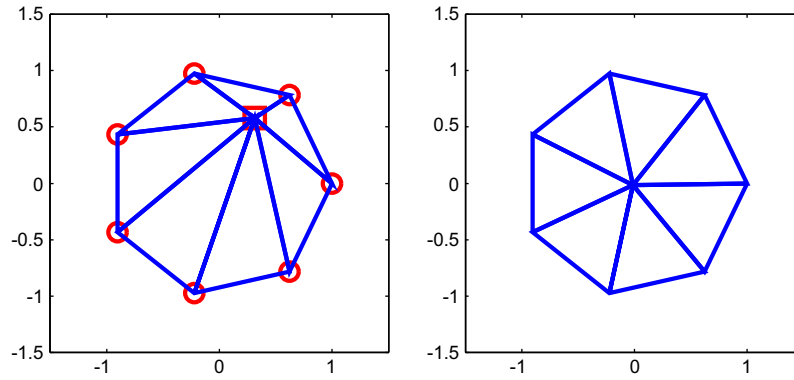


Fig. 5. Illustration of the mesh regularization procedure Eq. (7): p is moved to the center of the circle in order to balance the area of each triangle.

2.2. Additional ghost points

A small number of points located on a regular grid are added to the initial random points (lines 3–4). The regular grid is obtained by slightly enlarging in each directions the bounding box enclosing all mesh vertices and regularly adding points on each face of this grid (see Fig. 2 for an illustration in two dimensions).

We found experimentally that adding these points increases the quality of the resulting mesh. In all our experiments, we used 40 extra points in two-dimension and 600 extra points in three-dimension.

2.3. Removing external tetrahedrons

The Delaunay triangulation of a point-set covers the convex hull of the point-set. Tetrahedrons outside of the domain need to be peeled off (see Fig. 3 for an illustration in two-dimension). Distinguishing tetrahedrons inside and outside the volume to be meshed can be done by inspecting the sign of the function f at the centroid $\mathbf{c}$ of each tetrahedron: if $f(\mathbf{c}) < 0$, then the tetrahedron is outside, otherwise it is inside.

Practically, we may want to keep a slightly thicker solid. One possible approach is to discard tetrahedrons for which $f(\mathbf{c}) < -\epsilon$, for a small ϵ (we used 10^{-5} in all our experiments). Another approach is to compute the circumcenter $\mathbf{c}$ and the circumradius r of the tetrahedron t , and keep t if it is outside and if $d(\mathbf{c})/r < \alpha$, where α is a user defined constant (smaller than 1) and d is the distance from $\mathbf{c}$ to the surface. If f is not the signed distance function, then $d(\mathbf{c})$ can be approximated by $\|\mathbf{c} - \mathbf{c}_s\|$, where $\mathbf{c}_s$ is the projection of $\mathbf{c}$ on $f=0$ and is computed as explained in section 2.1.

2.4. Additional tetrahedron circumcenters

We select tetrahedron circumcenters $\mathbf{c}$ (line 8), where the ratio between the distance from $\mathbf{c}$ to the surface and the circumradius r is less than a given threshold (we used 0.2 in our experiments). This heuristic is trying to capture undersampled areas, which appear at the first iterations of the algorithm, when the number of points is small. Fig. 4 illustrates this type of configuration. The selected circumcenters are then projected on the surface and added to the list of additional points.

2.5. Mesh optimization

A triangular mesh approximating the surface $f=0$ is computed in line 11. However, the triangles are not necessarily well shaped

and the sharp features of the surface may not have been extracted correctly.

Optimization of the mesh is done by using an iterative scheme similar to the methods described in [13,12]. This scheme is summarized in Algorithm 2.

Algorithm 2. Mesh optimization

- 1: Move the triangle vertices to align the triangle normal with ∇f the gradient of f (see Section 2.5.1).
- 2: Regularize the shape of the triangles by vertex relocation (see Section 2.5.2).
- 3: Project the vertices $\mathbf{p}_i$ back on the surface $f=0$ (see Section 2.1).

2.5.1. Sharp features extraction

There are various existing methods for extracting the sharp features of the object, one possibility is the method proposed by Ohtake et al. in [13], where mesh vertices are moved in order to align the triangle normals with the gradient of f . Other possible approaches consist in optimizing vertex positions using a quadric error metric similar to the one described in [44]. This approach was taken by Ju et al. in [17] and Ohtake et al. in [12].

Experimentally, we obtained good results with the method from [13] which we briefly summarize here. Let $\mathbf{n}(T)$ be the normal to the triangle T , $\mathbf{m}(x,y,z) = \frac{\nabla f(x,y,z)}{\|\nabla f(x,y,z)\|}$ be the unit gradient to f , and $A(T)$ the area of the triangle T . The current vertex location $\mathbf{p}$ is updated as follow:

$$\mathbf{p}_{new} = \mathbf{p} + \frac{1}{\sum_{T_i \in Inc(\mathbf{p})} A(T_i)} \sum_{T_i \in Inc(\mathbf{p})} A(T_i) \mathbf{v}(T_i) \quad (6)$$

where $\mathbf{v}(T_i) = (\overline{p\mathbf{c}} \cdot \mathbf{m}(\mathbf{c})).\mathbf{m}(\mathbf{c})$ is the projection of $\overline{p\mathbf{c}}$ on the $\mathbf{m}(\mathbf{c})$ direction, $\mathbf{c}$ is the centroid of T_i and $Inc(\mathbf{p})$ corresponds to the set of triangles $\{T_i\}$ incident to the vertex $\mathbf{p}$.

In addition to retrieving sharp features, this step is improving the location of the mesh vertices as well.

2.5.2. Mesh regularization

During the mesh regularization step, the modified position $\mathbf{p}_{new}$ of a mesh vertex $\mathbf{p}$ is obtained by:

$$\mathbf{p}_{new} = \frac{1}{\sum_{T_i \in Inc(\mathbf{p})} A(T_i)} \sum_{T_i \in Inc(\mathbf{p})} A(T_i) \mathbf{c}(T_i) \quad (7)$$

where $Inc(\mathbf{p})$ corresponds to the set of triangles $\{T_i\}$ incident to the vertex $\mathbf{p}$, $A(T_i)$ is the area of the triangle T_i and $\mathbf{c}(T_i)$ is the centroid of the triangle T_i .

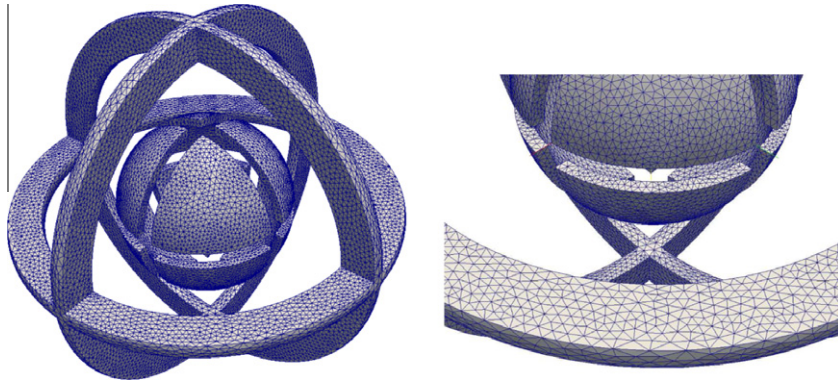


Fig. 6. Polygonization of a complex model ('core') with several disjoint components and sharp edges. Sharp features are properly retrieved (right). The surface mesh model contains 26,943 vertices.

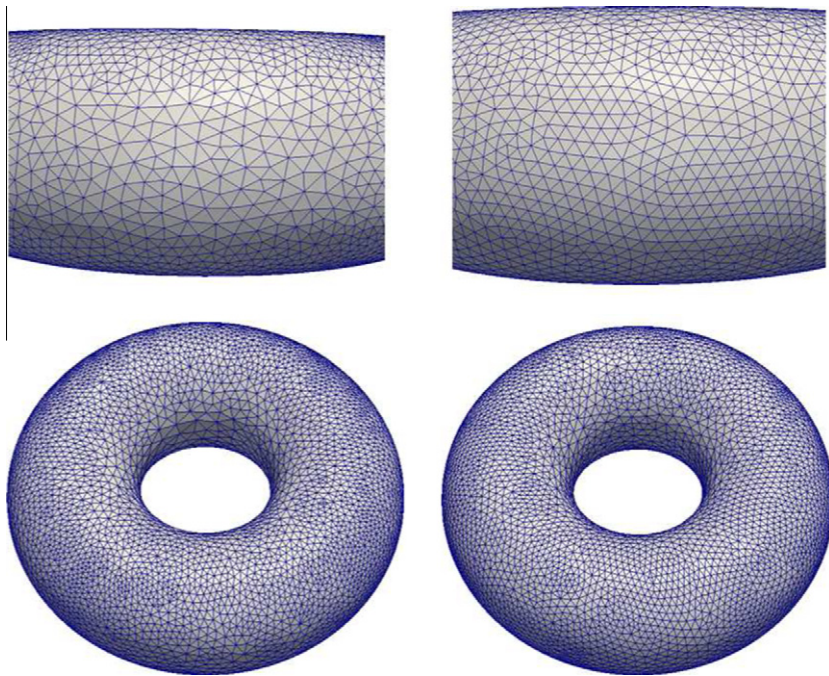


Fig. 7. Meshing of a torus with (right) or without (left) adaptive selection of vertices.

To illustrate Eq. (7), consider a set of planar triangles around a vertex $\mathbf{p}$, then applying Eq. 7 to $\mathbf{p}$ will move this point to balance the area of each triangle. Especially, if the triangle vertices (other than $\mathbf{p}$) lay on a circle, then $\mathbf{p}$ should be moved to the center of the circle as illustrated in Fig. 5.

Updating mesh vertices according to Eq. (7) may destroy sharp edges. It is preferable to identify sharp features and mark them as constraints, before invoking the mesh regularization procedure. Identifying sharp features is done by using a method proposed by Kobbelt et al. in [11]. A mesh vertex $\mathbf{p}$ is tagged as belonging to a sharp feature if: $\min_i(\mathbf{n}_i, \mathbf{n}_j) < \alpha$, where $\mathbf{n}_i$ is the gradient of f at the centroid $\mathbf{c}_i$ of the i -th triangle in the set of triangles incident to the vertex $\mathbf{p}$, and α a user controlled parameter (we used a value of 0.7 for α in our experiments).

2.5.3. Projection

Sharp feature extraction and mesh regularization may slightly move the mesh vertices away from the surface. In that case, the mesh vertices are projected back to the surface using the methods described in Section 2.1. The convergence of the methods described in section 2.1.1 or 2.1.2 is fast, since the vertices are initially already close to the surface.

2.6. Additional triangle centroids

We select a triangle centroid $\mathbf{c}$ (line 13), if its distance to the surface is greater than a given threshold and if the triangle circumradius r is greater than a given threshold (0.01 the minimum edge length of the bounding box in our experiments). Selecting triangle centroids in the area where the approximation error is high to properly adapt the surface approximation. Limiting the size of the circumradius allows to avoid too small triangles in the areas of high curvature. The selected triangle centroids are then projected on the surface and added to the list of points to be added.

2.7. Additional vertices

At the end of the main loop (lines 5–16) if the number of vertices is less than the user specified number n , new vertices are added in areas of high curvature (lines 17–22). An indicator for the surface curvature is computed at each triangle's centroid using the implicit surface function f (line 18). We compute this curvature indicator as: $A_t \|(\mathbf{n}_c - \mathbf{n}_1) + (\mathbf{n}_c - \mathbf{n}_2) + (\mathbf{n}_c - \mathbf{n}_3)\|$ where A_t is the area of the triangle t , $\mathbf{n}_c = -\frac{\nabla f(\mathbf{c})}{\|\nabla f(\mathbf{c})\|}$ is the normal at the centroid $\mathbf{c}$

of the triangle t (after projection on the surface) and $\mathbf{n}_i = -\frac{\nabla f(\mathbf{c}_i)}{\|\nabla f(\mathbf{c}_i)\|}$ are the normals at the centroids $\mathbf{c}_i$ (after projection on the surface) of the triangles neighbors to t (i.e. the triangles sharing an edge with t). Triangles are then inserted in a priority queue and sorted by increasing curvature (line 19). The triangle with highest curvature indicator is extracted and subdivided using a one to four subdivision (line 20). Neighboring triangles to the subdivided triangle are handled accordingly to avoid cracks and hanging vertices at each iteration of the loop. A triangle adjacent to only one subdivided triangle is split in two. A triangle adjacent to at least two subdivided triangles is subdivided using one to four subdivision. Finally the newly created triangles are added to the priority queue. These steps are repeated until the number of vertices in the triangle is greater than the user specified number of vertices n . After the addition of new vertices, we project again all vertices on the surface and optimize the triangle mesh with the techniques described in Section 2.5.

2.8. Mesh improvement

At line 23, the mesh may still contain a small number of skinny triangles with one or two very small angles. This will happen near the surface sharp features.

An additional step is added to iterate through the triangles and either swap or collapse edges for triangles with angles below some user defined threshold (we used a minimum angle of 15 in all our experiments).

This step may affect the quality of the mesh. Especially, edges on sharp features can be swapped or collapsed during the process described above. In order to prevent this, we relocate the mesh vertices by minimizing a quadric error metric as described in [12]. We alternate these two steps (edge swapping/collapsing and mesh optimization) into a loop that stops when all triangles have a minimum angle above the user given threshold or if a maximum number of iterations has been reached (a maximum of 10 iterations was used in our experiments).

2.9. Discussion

The adaptive algorithm improves the first algorithm by adding new vertices where the geometric approximation is insufficient instead of starting with a fixed number of randomly distributed points. This implies that the adaptive algorithm should create fewer triangles in low curvature areas and more triangles in high curvature areas. Given the maximum number of vertices, it does a better usage of the vertices budget.

3. Discretization of heterogeneous objects

In this section, we describe a novel algorithm, based on Algorithm 1, to extract surface meshes from implicitly represented heterogeneous models. A heterogeneous object is an object made of different constituent materials. Our algorithm converts implicitly defined heterogeneous objects into surface models separating the different homogeneous material regions.

3.1. Heterogeneous object modeling

Following the work described in [45], we model a heterogeneous object by an $(n+1)$ -tuple: $M = (G, A_1, \dots, A_n)$, where the geometry of the object G as well as the space partitions (or material regions) A_i for each material are defined by real valued functions: $G = \{x \in \mathbf{R}^3 : F(x) \geq 0\}$, $A_i = \{x \in \mathbf{R}^3 : S_i(x) \geq 0\}$, $i \in 1 \dots n$. We add the condition that the regions A_i form a partition of the solid G : $G = \bigcup_{i=1}^n A_i$. Each region has a unique material assigned to it: for

$i \neq j$, $A_i \cap A_j$ is either empty or the surface boundary between A_i and A_j .

We are interested in the discretization and the visualization of the heterogeneous object G surface with n materials assigned to the surface regions.

3.2. Algorithm for meshing of the heterogeneous object surface

Our meshing algorithm for heterogeneous objects contains two main steps: the first step is the meshing of each material region by using Algorithm 1 and the second step consists in assembling the meshes corresponding to each material region, while maintaining the detected boundaries between two adjacent materials. The pseudocode corresponding to this meshing algorithm is given in Algorithm 3.

Algorithm 3. Algorithm for meshing the surface of heterogeneous objects

```

1: for each space partition  $A_i (i \in \{1 \dots n\})$  do
2:   Compute a mesh  $(T_i, P_i)$  approximating the surface
    $S_i = 0$  with Algorithm 1.
3:   if  $A_i$  shares a boundary with another partition  $A_j (j > i)$ 
   then
4:     Compute the curve  $c_{ij} = \partial A_i \cap \partial A_j \cap \partial G$ .
5:     Regularly sample  $c_{ij}$ .
6:     Add these samples in a list of additional vertices  $L$ .
7:   end if
8:   Remove the vertices from  $P_i$  which are not on the
   surface of  $G$ .
9: end for
10: Compute  $B = \bigcup_j B_j$  the union of balls  $B_j = (x_j, r), x_j \in L$ .
11: Compute  $P = P_1 \cup \dots \cup P_n$ .
12: Remove points from  $P$  inside  $B$  (see the discussion
   below).
13: Compute the Delaunay triangulation  $D_t$  of  $P \cup X$ ,  $X = \{x_j\}_{j \in J}$ .
14: Remove tetrahedrons from  $D_t$  outside of  $G$ .
15: Identify  $T$  the faces of  $D_t$  on the surface of  $G$ .
```

3.3. Discussion of the heterogeneous object discretization algorithm

A surface mesh for each partition (line 2) in Algorithm 3 is computing using Algorithm 1. This step produces good quality triangle meshes corresponding to the surface of each material ($\partial A_i = \{(x, y, z) : S_i(x, y, z) = 0\}$). Meshing of each partition are independent tasks and can be run in their own thread.

The identification of the boundary curves between two materials (lines 3–4) is obtained from the functions defining the geometry of the object and the space partitions. Let S_i and S_j be the functions corresponding to the material regions A_i and A_j . The boundary curve between the surfaces $\{p : S_i(p) = 0\}$ and $\{p : S_j(p) = 0\}$ can be defined by $\{p : F(p) = (-S_i^2(p)) \wedge (-S_j^2(p)) = 0\}$, where $\wedge$ is the intersection operation implemented by an R-function [1]. Determining if a vertex p belongs to the boundary or not is done by evaluating the function $F(p)$ at that vertex. To improve the result, we can project each identified vertex along the gradient of $F(p)$ on the boundary curve.

After the identification of vertices on the boundary curve between two materials, we identify the edges connecting these vertices in the original mesh and subdivide these edges by inserting additional vertices (line 5).

These additional vertices, as well as the original vertices on the boundary curves between two materials, are maintained in a list of

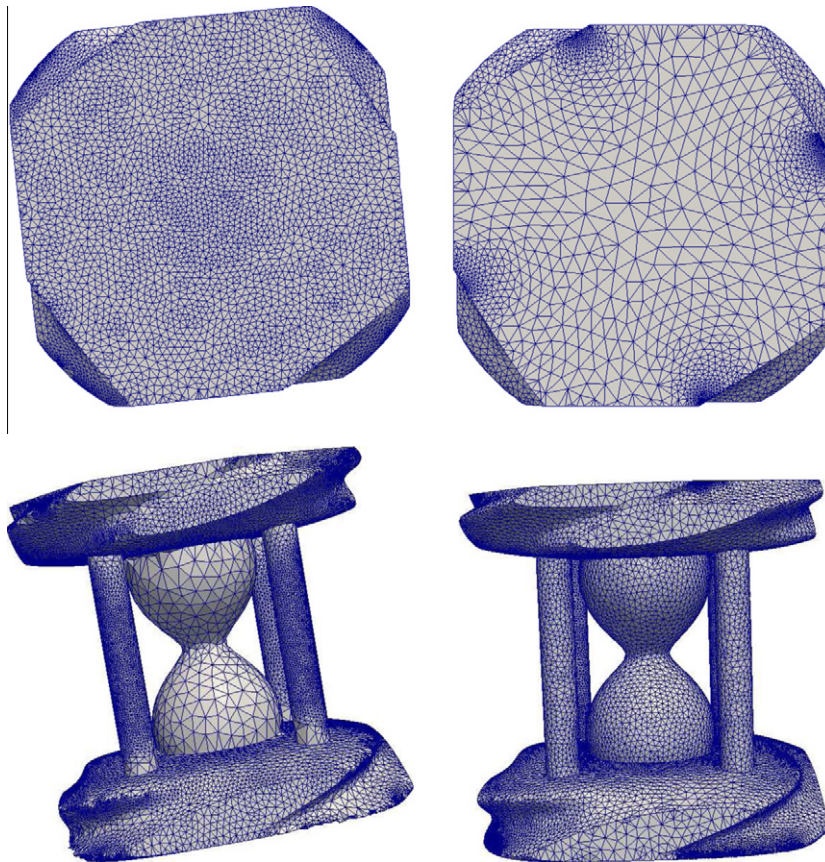


Fig. 8. Illustration of the vertex distribution with Algorithm 1 (right column) against the vertex distribution without adaptive selection of vertices (left column) for the model 'sand'. Vertices are more densely distributed in areas of high curvature with an adaptive selection of vertices.

Table 1

Number of vertices and faces, minimum and average minimum angles, approximation error and computation times for various models shown in the paper.

Model	#v	#Tris	Min/average minimum angle	Error	Time (s)
Sphere	3484	5764	28.3/52	1.4e−4	1.65
Torus	6815	12430	19.7/45	1.04e−4	2.9
Cylinders	13864	26528	20/46.5	1.25e−5	12.5
Core	26943	52678	16.9/46.3	3.6e−4	130
Sand	25856	50478	15.16/43	7.8e−4	170
Earth	30869	60606	18.92/43.5	7.1e−4	199

additional vertices (line 6). Points in this list are used as the centers of protecting balls (line 10). The union of these balls is formed (line 10) and the points corresponding to vertices of the surface meshes of each material that are inside this union of balls are discarded (lines 11–12). The radius of the balls r controlling the distance between the centers can be chosen as the minimum length of the triangle edges over all the material meshes. It is also possible to refine the subdivision algorithm (line 5) by selecting a subdivision size locally: given an edge on the material boundary, we identify the triangles sharing this edge, and take for r_i the minimum edge length for this triangle.

To improve the final result of Algorithm 3, it is possible to check if there is any edge in the final surface mesh that crosses the internal boundary between two materials, and project on the boundary the closer endpoint of the edge.

4. Experimental results

We present results that demonstrate the viability and effectiveness of our algorithms. Several examples of implicit surfaces and implicitly defined heterogeneous objects are meshed with our

algorithms. All computed surface meshes are rendered using flat shading.

The algorithms presented in this work have been implemented in *MATLAB™*. The computation of the Delaunay triangulation of a point-set is performed by Tetgen [46]. The computer used in the experiments was a Sun workstation, with an UltraSPARC IIIi processor at 1.5 GHz and 1 GB of main memory.

4.1. Results of the meshing algorithms

We first demonstrate that Algorithm 1 can produce surface mesh for complicated shapes as illustrated by Fig. 6. This object contains multiple disjoint components: the small sphere at the center of the object, the eight spherical parts around the core sphere and the outside torii. This object contains also several sharp features which can be seen on the right image in Fig. 6. The resulting surface mesh contains approximately 27,000 vertices.

Figs. 7 and 8 illustrate a torus and a more complex model meshed without adaptive selection of vertices (left) and with Algorithm 1 (right). Meshing without adaptive selection is done by generating

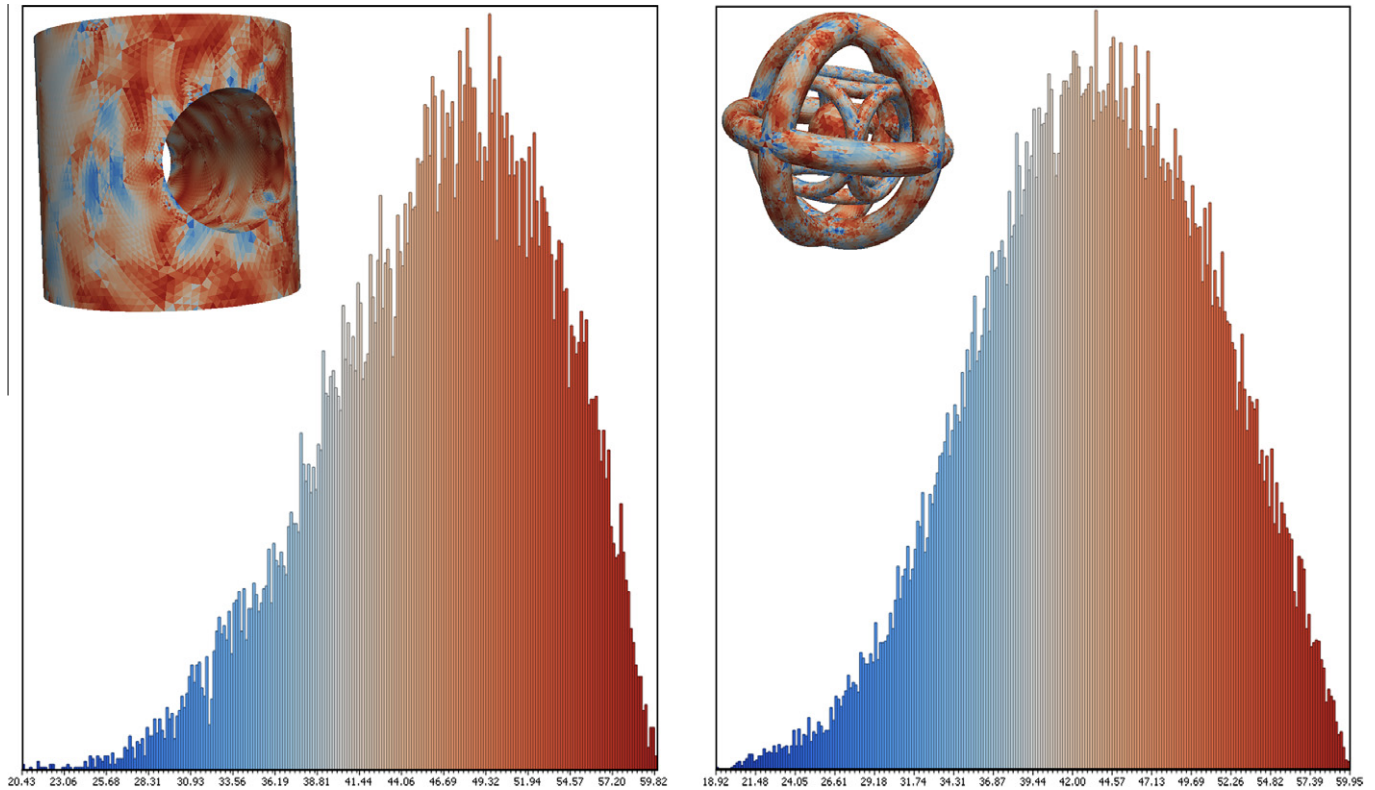


Fig. 9. Visual and mesh quality of the triangle mesh approximation of the models cylinders and earth.

Table 2

Number of vertices and faces, minimum and average minimum angles and approximation error for some of the previous models meshed with the algorithms discussed in [22], in [28] and in [34].

Model	#v	#Tris	Min/average minimum angle	Error
Macet cylinders	17308	34616	10.9/38.71	0.12
Macet core	22066	44120	17.16/40.23	0.019
Afront cylinders	6010	12020	13.8/49.67	0.15
Afront core	13659	27306	10.3/50.76	0.05
CGAL cylinders low	3238	6476	30/45.9	0.0017
CGAL cylinders	82126	164248	29.99/45.9	2.4e−4

n random points, projecting them on the surface (as described in Section 2.1) and applying the mesh optimization techniques described in Section 2.5. Adaptive insertion of vertices gives in general better results as illustrated in Fig. 8: vertex distribution is denser in areas of high curvature.

Table 1 shows quantitative results of Algorithm 1 on various models. The results are given in terms of the minimum and the average minimum angle over all triangles, the approximation error (computed as the sum of the squared distance error at each triangle centroid) and computation time (in seconds). The distance error at a point is approximated by: $e(p) \approx \frac{\|f(p)\|}{\|\nabla f(p)\|}$ where f is the implicit surface being meshed.

Triangle meshes for the models core, torus and sand were shown in Figs. 6–8. Triangle meshes approximating the surfaces of the models cylinders and earth are shown in Fig. 9. Each triangle is colored according to the minimum value of its three angles. The color ranges from blue¹ for the smallest minimum angle to red for the highest minimum angle. Distribution of the minimum angles are also shown on the right of each model.

We compare Algorithm 1 with existing algorithms for meshing implicit surfaces: macet [21,20,22], afront [28] and the algorithm discussed in [34]. Table 2 shows quantitative results of these algorithms applied to some of the previous models.

Macet and afront takes as input a three dimensional grid of scalar values. We used regular grids of $100 \times 100 \times 100$ scalar values obtained by sampling the implicit surface functions. The low resolution mesh for the model made of cylinders was obtained by using the CGAL surface mesher [47] with an angular bound of 30 and a distance bound of 0.01. The mesh with higher resolution was obtained by using a distance bound of 0.001. None of these algorithms retrieve properly sharp features of the models as illustrated in Fig. 10. In comparison, our algorithm can generate meshes maintaining the sharp features of the original model while producing triangles of similar quality.

Finally, we demonstrate with Fig. 11 that our algorithm can mesh complex freeform models.

4.2. Results of the algorithm for the polygonization of heterogeneous objects

We show results of applying Algorithm 3 for meshing the surfaces of several heterogeneous objects. The first example is a cube

¹ For interpretation of color in all figures, the reader is referred to the web version of this article.

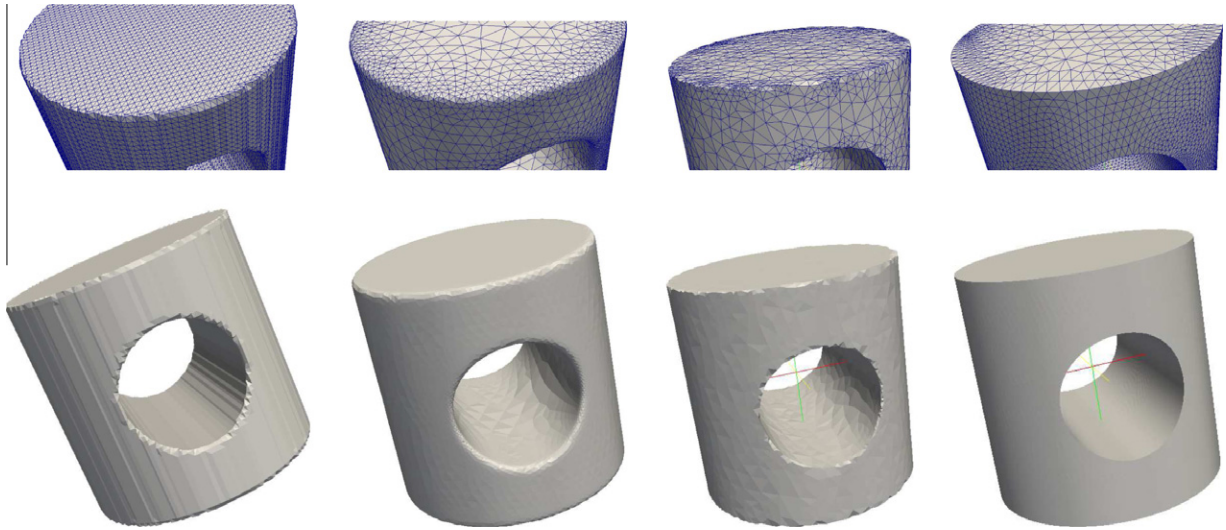


Fig. 10. Visual results of the cylinders meshed with the algorithms described in [22], in [28], in [34] and ours. The top row corresponds to zooms on a sharp feature of the object.

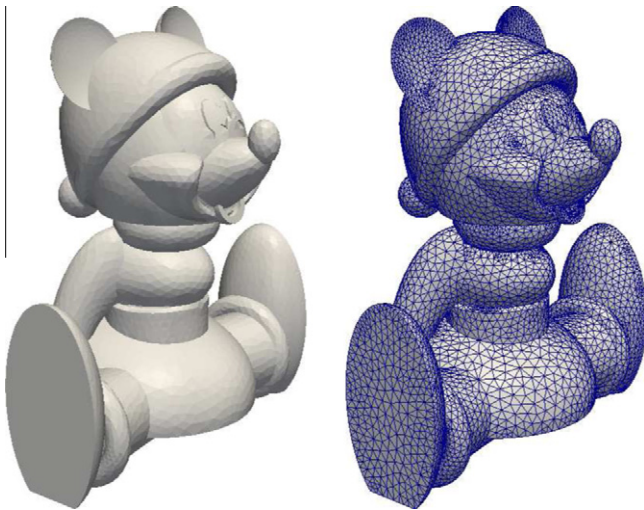


Fig. 11. Meshing of a freeform object.

made of three different homogeneous materials. The shape of each material region (space partition) is illustrated in Fig. 12. Material 1 is located in a half-torus (the torus is cut by the top face of the cube). Material 2 is defined in a volume bounded by a quarter of a sphere (the intersection of a sphere with three planes passing through its origin). Finally, the third material is located in the

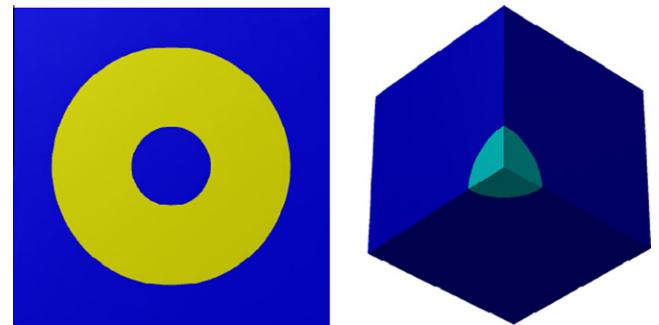


Fig. 13. Discretization using our algorithm of the surface of a heterogeneous solid made of three homogeneous materials. Different views of the faces of the solid are displayed from left to right.

remaining space, i.e. the cube with subtracted space partitions of materials 1 and 2.

The final multi-material object is shown Fig. 13. Each material is assigned a different color: yellow for material 1, green for material 2 and blue for material 3. This mesh was generated in 45 seconds. It contains 6023 vertices and 10838 triangles. The approximation error (i.e. the sum of the squared error at each centroid) is $6.6e-5$.

The second example is a chess pawn. The geometry of the pawn can be constructed by a set-expression given in [4]. Three materials have been defined over the pawn as illustrated in Fig. 14. The

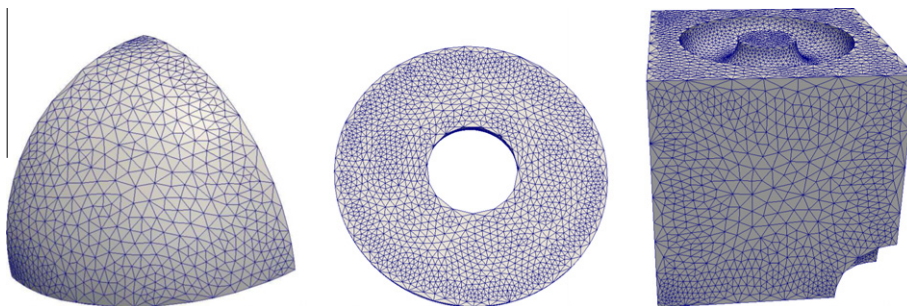


Fig. 12. Results of the polygonization of the surface of each material region. From left to right: region corresponding to Material 1-3.

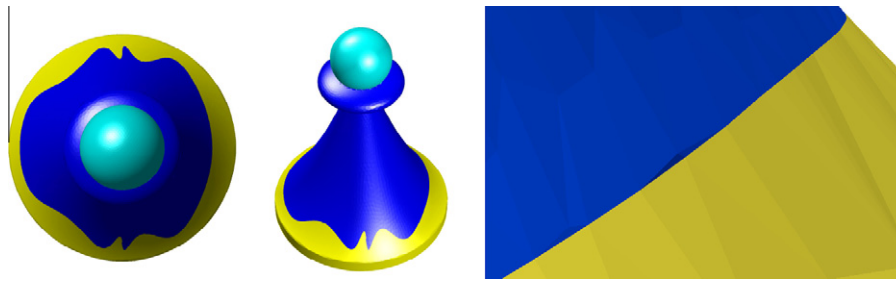


Fig. 14. A chess pawn made of three materials and meshed with our algorithm. From left to right: the final pawn with the three material regions (as seen from the top), another view of the pawn and a zoom on the sinusoidal boundary curve between material two and three.

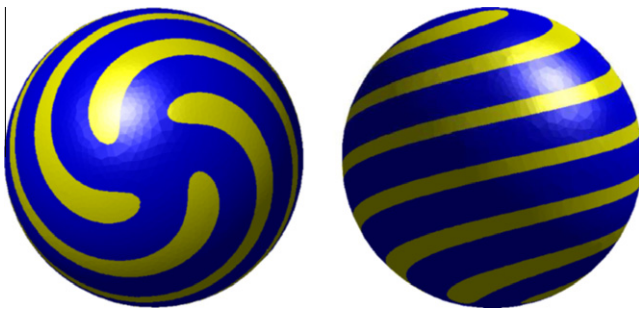


Fig. 15. A complex model made of two different homogeneous materials: the first one, in yellow, on the spirals and the second one, in blue, on the sphere. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

second material region is defined by a complex geometry obtained from the intersection of a modified cylinder, where the height is bounded by a sinusoidal function, with the bottom part of the pawn. Different views and a zoom on the boundary curve are illustrated in Fig. 14. It took 172 seconds to generate this mesh. The sum of the squared error at each triangle centroid is $1.1e-3$. The final mesh shown in Fig. 14 is made of 11,151 vertices and 22,298 triangles.

The final example, in Fig. 15, is made of two homogeneous materials defined on a sphere. The first material region, in yellow, corresponds to the union of several spirals. The second material region, in blue, corresponds to the rest of the sphere.

4.3. Limitations

The proposed algorithms rely on computing the Delaunay triangulation of a point-set. In our experiments, we used Tetgen [46], but other implementations exist and may be used as well such as the one available in CGAL [47].

One bottleneck of the algorithms is the number of evaluations of the function defining the solid (and the material regions). If the gradient of the function is unknown, it also has to be computed numerically by finite differences (which means three additional function evaluations). Function evaluation is needed when determining if a tetrahedron is inside or outside the solid, when projecting triangle centroids and tetrahedron circumcenters to the surface, and at each step during the mesh optimization (for retrieving sharp features and for projecting the vertices on the surface). In total, there are four function calls at each projection and there are three projections at each step (one projection for the triangle centroids, one projection for the tetrahedron circumcenters and one projection at each mesh optimization).

The triangles produced by our algorithm for meshing the surface of multi-material objects near the interfaces between two or more materials exhibit a lower quality than the rest of the triangles. This degradation is caused by the additional vertices added

along the curved boundaries and necessary for keeping the edges of the triangle mesh aligned with the material boundaries.

5. Conclusion

We have proposed in this paper a new algorithm for meshing implicit surfaces that generate good quality triangle meshes. We have illustrated this algorithm by several practical examples, including complex implicit surfaces with several disconnected components and sharp features, and freeform objects. Using this algorithm, we have proposed a new algorithm for meshing surfaces of heterogeneous objects. This algorithm maintains the curved boundaries between different materials, while producing good quality triangle meshes.

Acknowledgments

We would like to thank the anonymous reviewers for their comments and suggestions. The models for 'core', 'sand' and 'mouse' are from the HyperFun project (<http://www.hyperfun.org/wiki/doku.php?id=gallery:main>).

References

- [1] Pasko A, Adzhiev V, Sourin A, Savchenko V. Function representation in geometric modeling: concepts, implementation and applications. *Visual Comput* 1995;8(11):429–46.
- [2] Bloomenthal J, editor. *Introduction to implicit surfaces*. Morgan-Kaufmann; 1997.
- [3] Osher S, Fedkiw R. *Level set methods and dynamic implicit surfaces*. Springer; 2002.
- [4] Shapiro V. Semi-analytic geometry with r-functions. *Acta Numer* 2007;16: 239–303.
- [5] Lorensen W, Cline H. Marching cubes: a high resolution 3d surface construction algorithm. *Comput Graph* 1987;21(4):163–9.
- [6] Newman T, Yi H. A survey of the marching cubes algorithm. *Comput Graph* 2006;30:854–79.
- [7] Natarajan BK. On generating topologically consistent isosurfaces from uniform samples. *Visual Comput* 1994;11(1):52–62.
- [8] Chernyaev E. *Marching cubes 33: Construction of topologically correct isosurfaces*. Tech. rep., CERN; 1995.
- [9] Lewiner T, Lopes H, Vieira AW, Tavares G. Efficient implementation of marching cubes' cases with topological guarantees. *J Graph Tools* 2003;8(2): 1–15.
- [10] Pasko A, Pilyugin V, Pokrovskiy V. Geometric modeling in the analysis of trivariate functions. *Comput Graph* 1988;12:457–65.
- [11] Kobbelt L, Botsch M, Schwanke U, Seidel H-P. Feature sensitive surface extraction from volume data. In: *SIGGRAPH'01 Proceedings*; 2001. p. 57–66.
- [12] Ohtake Y, Belyaev A. Dual/primal mesh optimization for polygonized implicit surfaces. In: *ACM solid modeling proceedings*; 2002. p. 171–8.
- [13] Ohtake Y, Belyaev A, Pasko A. Dynamic mesh optimization for polygonized implicit surfaces with sharp features. *Visual Comput* 2003;19(2–3):115–26.
- [14] Wilhelms J, Gelder AV. Octrees for faster isosurface generation. *ACM Trans Graph* 1992;11(3):201–27.
- [15] Shu R, Chen Z, Kankanhalli MS. Adaptive marching cubes. *Visual Comput* 1995;11(4):202–17.
- [16] Shekhar R, Fayyad E, Yagel R, Cornhill JF. Octree-based decimation of marching cubes surfaces. In: *Proceedings of IEEE Visualization 1996, IEEE*; 1996. p. 335–42.
- [17] Ju T, Losasso F, Schaefer S, Warren J. Dual contouring on hermite data. In: *ACM Siggraph 2002 proceedings*; 2002. p. 339–46.

- [18] Ju T, Udeshi T. Intersection-free contouring on an octree grid. In: Pacific graphics 2006 proceedings; 2006.
- [19] Ho C-C, Wu F-C, Chen B-Y, Chuang Y-Y, Ouhyoung M. Cubical marching squares: adaptive feature preserving surface extraction from volume data. *Comput Graph Forum* 2005;24(3).
- [20] Dietrich C, Scheidegger C, Comba J, Nedel L, Silva C. Edge transformations for improving mesh quality of marching cubes. *IEEE Trans Visual Comput Graph* 2009;15(1):150–9.
- [21] Dietrich C, Scheidegger C, Comba J, Nedel L, Silva C. Edge groups: an approach to understanding the mesh quality of marching methods. *IEEE Trans Visual Comput Graph* 2008;14(6):1651–8.
- [22] Dietrich C, Scheidegger C, Comba J, Nedel L, Silva C. Marching cubes without skinny triangles. *Comput Sci Eng* 2009;11:82–7.
- [23] Raman S, Wenger R. Quality isosurface mesh generation using an extended marching cubes lookup table. *Comput Graph Forum* 2008;27(3):791–8.
- [24] Wyvill G, McPheeters C, Wyvill B. Data structures for soft objects. *Visual Comput* 1986;2(4):227–34.
- [25] Hilton A, Stoddart A, Illingworth J, Windeatt T. Marching triangles: range image fusion for complex object modelling. In: International conference on image processing proceedings; 1996. p. 381–4.
- [26] Boissonnat J-D. Geometric structures for three-dimensional shape representation. *ACM Trans Graph* 1984;3(4):266–86.
- [27] Akkouchi S, Galin E. Adaptive implicit surface polygonization using marching triangles. *Comput Graph Forum* 2001;20(2):67–80.
- [28] Schreiner J, Scheidegger C, Silva C. High-quality extraction of isosurfaces from regular and irregular grids. *IEEE Trans Visual Comput Graph* 2006;12(5):1205–12.
- [29] Desbrun M, Tsingos N, Gascuel M-P. Adaptive sampling of implicit surfaces for interactive modeling and animation. *Comput Graph Forum* 1996;15(5):319–25.
- [30] L.H. De Figueiredo, J. De Miranda Gomes, D. Terzopoulos, L. Velho, Physically-based methods for polygonization of implicit surfaces. In: Graphics interface 92 proceedings; 1992. p. 250–7.
- [31] A. Witkin, P. Heckbert, Using particles to sample and control implicit surfaces. In: ACM Siggraph 1994 proceedings; 1994. p. 260–8.
- [32] Meyer M, Kirby R, Whitaker R. Topology, accuracy and quality of isosurface meshes using dynamic particles. *IEEE Trans Visual Comput Graph* 2007;13(6):1704–11.
- [33] Chew I.P. Guaranteed-quality mesh generation for curved surfaces. In: Proceedings of the ninth symposium on computational geometry. ACM; 1993. p. 274–80.
- [34] Boissonnat J-D, Oudot S. An effective condition for sampling surfaces with guarantee, Tech. Rep. RR-5064, INRIA, December 2003.
- [35] Dey T, Levine J. Delaunay meshing of isosurfaces. *Visual Comput* 2008;24(6):411–22.
- [36] Edelsbrunner H, Shah NR. Triangulating topological spaces. In: Proceedings of the symposium on computational geometry. ACM; 1994. p. 285–92.
- [37] Gelas A, Valette S, Prost R, Nowinski W. Variational implicit surface meshing. *Comput Graph* 2009;33(3):312–20.
- [38] Valette S, Chassery J-M, Prost R. Generic remeshing of 3d triangular meshes with metric-dependent discrete voronoi diagrams. *IEEE Trans Visual Comput Graph* 2008;14(2):369–81.
- [39] Nelson G, Franke R. Computing the separating surface for segmented data. In: Visualization Conference, IEEE; 1997. p. 229–33. doi:<http://www.doi.ieee.org/10.1109/VISUAL.1997.663887>.
- [40] J. Bloomenthal, K. Ferguson, Polygonization of non-manifold implicit surfaces, in: SIGGRAPH '95: Proceedings of the 22nd annual conference on computer graphics and interactive techniques. New York (NY, USA): ACM; 1995. p. 309–16. doi:<http://www.doi.acm.org/10.1145/218380.218462>.
- [41] Suzuki H, Fujimori T, Michikawa T, Miwata Y, Sadaoka N. Skeleton surface generation from volumetric models of thin plate structures for industrial applications. In: Mathematics of surfaces XII. Lecture notes in computer science, vol. 4647. Berlin/ Heidelberg: Springer; 2007. p. 442–64.
- [42] Shammaa MH, Suzuki H, Ohtake Y. Extraction of isosurfaces from multi-material ct volumetric data of mechanical parts. In: SPM '08: Proceedings of the 2008 ACM symposium on solid and physical modeling. New York (NY, USA): ACM; 2008. p. 213–20. doi:<http://www.doi.acm.org/10.1145/1364901.1364931>.
- [43] Meyer M, Whitaker R, Kirby RM, Ledergerber C, Pfister H. Particle-based sampling and meshing of surfaces in multimaterial volumes. *IEEE Transactions on Visualization and Computer Graphics* 2008;14:1539–46. doi:<http://www.doi.ieeecomputersociety.org/10.1109/TVCG.2008.154>.
- [44] Garland M, Heckbert PS. Surface simplification using quadric error metrics. In: Computer graphics (proceedings of SIGGRAPH '97); 1997. p. 209–16.
- [45] Pasko A, Adzhiev V, Schmitt B, Schlick C. Constructive hypervolume modeling. *Graph Models* 2001;63(6):413–42.
- [46] Si H. TetGen, A Quality Tetrahedral Mesh Generator and Three-Dimensional Delaunay Triangulator. <<http://www.tetgen.berlios.de/>>.
- [47] Cgal, Computational Geometry Algorithms Library. <<http://www.cgal.org>>.